



UNIVERSIDAD NACIONAL EXPERIMENTAL DEL TÁCHIRA (UNET)

SAN CRISTÓBAL, TÁCHIRA, VENEZUELA

De la implementación física a la simulación

Migración del robot futbolista *FootBot* a un entorno
ROS 2 Humble + Gazebo Fortress

Reporte de entrega — Componente de Simulación

Autores: Chacón Gómez, José Daniel

Proyecto: FootBot / Auto Soccer Bot

Stack: ROS 2 Humble · Gazebo Fortress

Fecha: Junio de 2026

Índice

Resumen	2
1. Introducción	2
1.1. ¿Qué son ROS 2 y Gazebo?	3
1.2. Continuidad con el trabajo previo	3
2. Motivación: ¿por qué migrar a simulación?	4
3. Del robot físico al entorno simulado	5
4. Arquitectura del sistema de simulación	5
4.1. Stack tecnológico	6
4.2. Paquetes del <i>workspace</i>	6
4.3. Flujo de datos	6
5. Fases del trabajo realizado	7
5.1. Fase 1 — Fundamentos: modelo, mundos y arranque	7
5.2. Fase 2 — Puentes de control	7
5.3. Fase 3 — Percepción simulada	7
5.4. Fase 4 — Comportamientos autónomos deterministas	7
5.5. Fase 5 — Datasets y entrenamiento de visión	7
5.6. Mundos y escenarios	8
6. Comportamientos autónomos en detalle	8
6.1. Control de balón (Ball Control)	8
6.2. Llegar a la portería (Reach Goal)	9
7. Percepción y <i>datasets</i>	10
7.1. Cámaras y detección	10
7.2. Tubería de <i>dataset</i> y entrenamiento	11
8. Resultados y estado actual	12
8.1. Análisis experimental: Llegar a la portería (5 episodios)	13
9. Trabajo futuro	14
10. Conclusiones	15
Referencias	15

Resumen

El proyecto *FootBot* (también llamado *Auto Soccer Bot*) nació como un robot futbolista físico de bajo costo, construido alrededor de un ESP32-CAM, con la percepción y la toma de decisiones ejecutándose en un portátil. Este reporte documenta la **migración de ese sistema hacia un entorno de simulación** basado en **ROS 2 Humble** y **Gazebo Fortress**. Se explica *qué* se construyó, *por qué* se decidió pasar del hardware real a la simulación, *cómo* se trasladó cada componente del robot físico a su equivalente simulado, y *qué fases* se ejecutaron: fundamentos (modelo del robot y mundos), puentes de control, percepción simulada, comportamientos autónomos deterministas (*Control de balón* y *Llegar a la portería*) y la tubería de *datasets* y entrenamiento de visión. El resultado es un *workspace* de ROS 2 reproducible que conserva la compatibilidad con el robot ESP32 como referencia, a la vez que habilita el desarrollo seguro, repetible y escalable de comportamientos de fútbol robótico.

Palabras clave: robótica móvil, ROS 2 Humble, Gazebo Fortress, simulación, máquinas de estados finitos, percepción, YOLO, fútbol robótico.

1. Introducción

El sistema original, *Auto Soccer Bot*, es un robot móvil de tracción diferencial basado en un ESP32-CAM que opera en dos modos complementarios: (i) **control manual** mediante gestos de la mano detectados por visión (MediaPipe) en un portátil, y (ii) **modo automático**, en el que el ESP32-CAM transmite vídeo por Wi-Fi a una aplicación en el host que realiza detección del balón (umbrales HSV junto con YOLO) y alimenta una máquina de estados finitos (FSM) para seguir el balón. Todos los comandos viajan por una API HTTP (/move) desacoplada del flujo de vídeo.

Ese diseño valida la arquitectura distribuida (percepción pesada en el portátil, actuación en tiempo real en el robot), pero arrastra las limitaciones propias de cualquier banco de pruebas físico: dependencia del hardware, de la iluminación y de un único robot, fragilidad mecánica, y ciclos de iteración lentos. Para avanzar hacia comportamientos de fútbol más ambiciosos —posesión del balón, conducción hacia portería, oponentes y juego en equipo— se decidió construir una **capa de simulación** sobre ROS 2 y Gazebo.

Este documento reporta esa migración. La capa de simulación vive aislada bajo el directorio /simulation y mantiene intactas las aplicaciones ESP32 y Python originales como referencia. El resto del reporte se organiza así: la Sección 2 expone la motivación; la Sección 3 describe cómo se trasladó cada componente físico a su equivalente simulado; la Sección 4 presenta la arquitectura del sistema de simulación; la Sección 5 detalla las fases ejecutadas; las Secciones 6 y 7 profundizan en los comportamientos autónomos y en la percepción; la Sección 8 resume el estado actual y la Sección 9 el trabajo futuro.

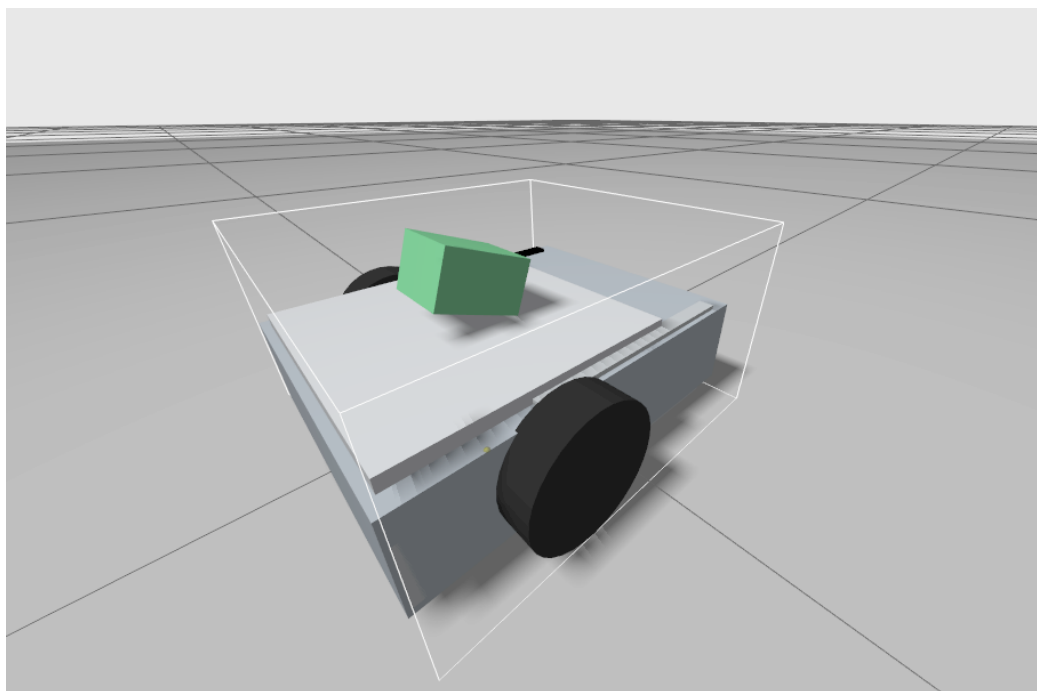


Figura 1: FootBot generado en Gazebo con la cámara simulada del robot, la iluminación y los objetos de validación visibles. La simulación reproduce el robot y su sensor de cámara dentro de un mundo controlado y repetible.

1.1 ¿Qué son ROS 2 y Gazebo?

ROS 2 (*Robot Operating System 2*) no es un sistema operativo, sino un *middleware* de robótica: un conjunto de librerías y convenciones para construir el sistema como un grafo de procesos independientes —los *nodos*— que se comunican publicando y suscribiéndose a canales con nombre, los *topics*. Aporta mensajes tipados, parámetros, lanzamiento coordinado y descubrimiento automático, lo que favorece un diseño modular y reutilizable.

Gazebo (edición *Fortress*) es un *simulador* de robótica 3D: modela física (gravedad, colisiones, fricción), sensores (cámaras, odometría) y mundos completos, y permite generar (*spawn*) un robot para observar su comportamiento sin hardware. La pieza que une ambos mundos es el `ros_gz_bridge`, que traduce los datos entre Gazebo y los *topics* de ROS 2; gracias a él, los nodos de percepción y control operan igual frente a un robot real o uno simulado.

1.2 Continuidad con el trabajo previo

Este trabajo da continuidad al reporte anterior del *Auto Soccer Bot*, centrado en el robot físico ESP32-CAM de control híbrido. Aquel sistema ya logró: (i) teleoperación manual por gestos de la mano (MediaPipe); (ii) un modo automático de **seguimiento del balón** con percepción híbrida HSV+YOLO y una FSM; y (iii) el entrenamiento y validación de detectores YOLO de **goal** y **opponent** ($\text{mAP}@0.5 = 0.991$). Sin embargo, esos detectores de portería y oponente **no llegaron a integrarse** en la FSM del modo automático: la fusión de decisiones multiobjeto quedó explícitamente como trabajo futuro.

¿Qué cambió de forma notable al pasar a la simulación?

- **Plataforma.** Del robot ESP32-CAM físico a un FootBot simulado en Gazebo (modelo URDF de tracción diferencial); el *stream* MJPEG pasa a ser un *topic* de cámara.

- **Comunicación y latencia.** Del enlace Wi-Fi/HTTP —sensible a la latencia, que obligaba a retención del último frame y a limitar la tasa de comandos— a *topics* de ROS 2 en un grafo local, con latencia mucho menor y estable (Sección 2).
- **La portería, por fin integrada.** El comportamiento *Llegar a la portería* fusiona `ball + goal` en el lazo percepción→decisión y conduce el balón a la portería: la fusión multiobjeto que el reporte previo dejaba pendiente **ya está parcialmente realizada** (resta el oponente).
- **Ingeniería.** De aplicaciones monolíticas en el host a **paquetes ROS 2 modulares**, con un patrón determinista percepción→estado→FSM, una suite de pruebas y un *árbitro* de simulación para puntuar goles.

2. Motivación: ¿por qué migrar a simulación?

La migración a un entorno simulado no busca reemplazar al robot físico, sino crear un banco de pruebas donde la lógica de fútbol pueda desarrollarse y validarse de forma **rápida, segura y reproducible**. Las razones principales fueron:

- **Reproducibilidad y determinismo.** Cada escenario (posición del balón, de la portería, del robot) se reinicia idéntico, lo que permite comparar cambios de comportamiento sin el ruido del mundo físico.
- **Seguridad y costo.** No hay motores que quemar, baterías que agotar ni colisiones que reparar; iterar es gratis y no desgasta hardware.
- **Velocidad de iteración.** No es necesario montar el robot ni el campo físico para probar una idea: se lanza un *world* y se ejecuta el comportamiento.
- **Escenarios paralelos y casos límite.** La simulación permite ejecutar varios escenarios aislados a la vez (p. ej., balón al frente, lejos y detrás) en un mismo mundo, algo inviable con un solo robot real.
- **Independencia del hardware y la iluminación.** La cámara simulada entrega imágenes consistentes, eliminando la fragilidad del detector frente a cambios de luz que afectaba al lazo físico.
- **Latencia menor y estable.** En el robot físico, el enlace Wi-Fi/HTTP entre el portátil y el ESP32 añadía un retardo variable al lazo de control; en simulación, la comunicación por *topics* locales reduce y estabiliza esa latencia.
- **Base para comportamientos avanzados.** Posesión del balón y conducción hacia portería requieren muchísimas repeticiones; la simulación las hace prácticas.
- **Continuidad.** El trabajo previo no se descarta: el robot ESP32 y sus apps Python quedan como referencia, y un puente mantiene la compatibilidad del antiguo protocolo HTTP /move.

Latencia y compatibilidad HTTP

Una limitación importante del montaje físico era la **latencia variable del enlace Wi-Fi** entre el portátil y el ESP32: tanto el vídeo como los comandos viajaban por HTTP, lo que introducía retardo y obligaba a paliativos como la retención del último frame y la li-

mitación de la tasa de comandos (el robot llegaba a *perseguir el pasado*). En la simulación, la comunicación ocurre por *topics* de ROS 2 dentro de un mismo grafo local, de modo que esa latencia se **reduce drásticamente y se estabiliza**.

¿Conservamos el modo de operar por HTTP? **Sí, pero solo por compatibilidad:** el paquete `footbot_bridge` mantiene vivo el antiguo endpoint HTTP `/move` traduciéndolo a `/cmd_vel`. El control nativo y recomendado dentro de la simulación es directamente por el *topic* `/cmd_vel`.

Regla de oro del control

En cualquier modo de lanzamiento, **un solo nodo** debe ser dueño de `/cmd_vel` a la vez. Esta regla, heredada del diseño original, evita que controladores en competencia se peleen por el robot y es un principio transversal de toda la arquitectura de simulación.

3. Del robot físico al entorno simulado

La estrategia fue *simulation-first*: en lugar de portar el firmware del ESP32, se reconstruyó cada capacidad del robot como uno o varios paquetes de ROS 2, reutilizando los conceptos (tracción diferencial, cámara, detección de balón, FSM) pero apoyándose en la infraestructura nativa de ROS y Gazebo. La Tabla 1 resume la correspondencia.

Tabla 1: Correspondencia entre el robot físico y su equivalente simulado.

Componente del robot físico	Equivalente en simulación (ROS 2 + Gazebo)
Chasis y tracción diferencial (motores DC, driver L298N)	Modelo URDF/Xacro (<code>footbot_description</code>) accionado por el plugin DiffDrive de Gazebo (<code>footbot_gazebo</code>)
Cámara ESP32-CAM (stream MJPEG por Wi-Fi)	Cámara simulada de Gazebo, puenteada a <code>/camera/image_raw</code> por <code>ros_gz_bridge</code>
API HTTP <code>/move</code> del firmware ESP32	Puente <code>footbot_bridge</code> (HTTP <code>/move</code> → <code>/cmd_vel</code>) y control nativo por <code>/cmd_vel</code>
Control manual por gestos (MediaPipe en el host)	<code>footbot_perception</code> (detección de manos) + <code>footbot_control</code> (<code>gesture_to_cmd_vel</code>)
Detección de balón por color HSV (host)	<code>footbot_perception</code> (<code>ball_detector</code>)
Modelos YOLO de portería/oponente (<code>soccer_vision</code>)	<code>footbot_soccer_vision</code> (detectores YOLO, captura y herramientas de <i>dataset</i>)
FSM de seguimiento de balón (host)	<code>footbot_soccer_behavior</code> (<code>ball_control_fsm</code> , <code>reach_goal_fsm</code> , estimadores de estado y <i>skills</i>)
Archivos de configuración de Python	Parámetros de ROS en archivos YAML
Red Wi-Fi local (host ↔ robot)	Topics de ROS 2 dentro del grafo de nodos

En esencia: lo que antes era un *stream* MJPEG hacia un detector y un POST HTTP hacia el firmware, ahora es un *topic* de cámara hacia un nodo de percepción y un *topic* `/cmd_vel` hacia el robot simulado. El antiguo protocolo `/move` no se abandona: el paquete `footbot_bridge` lo traduce a `/cmd_vel`, conservando compatibilidad con el ecosistema ESP32.

4. Arquitectura del sistema de simulación

4.1 Stack tecnológico

El entorno soportado es deliberadamente el oficial y de bajo riesgo para Ubuntu 22.04:

- **ROS 2 Humble Hawksbill** como middleware de robótica.
- **Gazebo Fortress** como simulador físico/visual.
- **ros-humble-ros-gz** como integración oficial ROS ↔ Gazebo.
- **Ubuntu 22.04 (jammy)** y **Python 3.10**.

Se descartó ROS 2 Jazzy/Gazebo Harmonic en este host porque apuntan a Ubuntu 24.04; mantener el emparejamiento por defecto reduce el riesgo y evita soluciones alternativas de versión.

4.2 Paquetes del *workspace*

El *workspace* de ROS2 vive en `simulation/ros2_ws/` y se divide en paquetes con responsabilidades acotadas (Tabla 2).

Tabla 2: Paquetes del *workspace* de simulación y su responsabilidad.

Paquete	Tipo	Responsabilidad
<code>footbot_common</code>	<code>ament_python</code>	Constantes compartidas (topics, frames, geometría, utilidades).
<code>footbot_description</code>	<code>ament_cmake</code>	Modelo Xacro del robot, frames, ruedas, cámara, RViz.
<code>footbot_gazebo</code>	<code>ament_cmake</code>	Mundos, modelos, configuración del puente, plugin de arrastre del balón.
<code>footbot_bringup</code>	<code>ament_cmake</code>	Orquestación de lanzamiento de todos los modos.
<code>footbot_bridge</code>	<code>ament_python</code>	Puente HTTP /move (compatible ESP32) hacia /cmd_vel.
<code>footbot_perception</code>	<code>ament_python</code>	Webcam, detección de manos, detección HSV del balón, visor de depuración.
<code>footbot_control</code>	<code>ament_python</code>	Controladores gesto → velocidad y seguidor de balón simple.
<code>footbot_soccer_msgs</code>	<code>ament_cmake</code>	Mensajes propios (<code>BallState</code> , <code>BallGoalState</code>).
<code>footbot_soccer_behavior</code>	<code>ament_python</code>	Estimación de estado, <i>skills</i> y FSM de los comportamientos.
<code>footbot_soccer_vision</code>	<code>ament_python</code>	Percepción YOLO, captura y herramientas de <i>dataset</i> .

4.3 Flujo de datos

El patrón general es: Gazebo publica sensores (cámara, odometría) por Gazebo Transport, el `ros_gz_bridge` los traduce a topics de ROS, los nodos de percepción producen detecciones, los estimadores las convierten en estado, y las FSM deciden y publican /cmd_vel (Fig. 2).

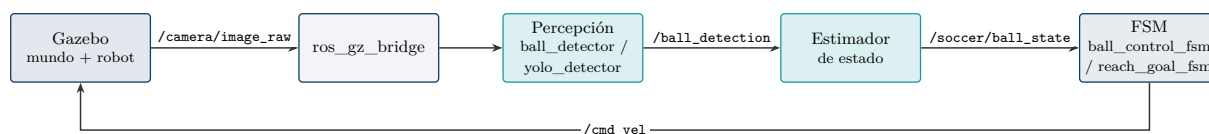


Figura 2: Arquitectura abstracta y flujo de *topics*: Gazebo entrega la imagen de la cámara, el puente la traduce a ROS, la percepción detecta, el estimador produce el estado y la FSM cierra el lazo publicando `/cmd_vel`. Se muestran solo los *topics* principales.

Tubería de *Control de balón*

```

/camera/image_raw → ball_detector → /ball_detection → ball_state_estimator
→ /soccer/ball_state → ball_control_fsm → /cmd_vel

```

Tubería de *Llegar a la portería*

```

/camera/image_raw → yolo_detector → /soccer/detections →
ball_goal_state_estimator → /soccer/ball_goal_state → reach_goal_fsm
→ /cmd_vel

```

5. Fases del trabajo realizado

5.1 Fase 1 — Fundamentos: modelo, mundos y arranque

Se decidió el *stack* (ROS 2 Humble + Gazebo Fortress), se creó el *workspace* de *colcon*, y se construyó el modelo del robot en Xacro (*footbot_description*): chasis, ruedas, rueda loca, sensor de cámara y frames (*base_footprint*, *base_link*, *camera_link*, etc.). Se prepararon los mundos de Gazebo y el lanzamiento base, que genera (*spawn*) el robot y púentea `/cmd_vel`, `/odom` y los *topics* de cámara.

5.2 Fase 2 — Puentes de control

Se habilitaron las tres vías de control: comandos directos por `/cmd_vel`; el puente *footbot_bridge*, que mantiene el antiguo endpoint HTTP `/move` del ESP32 traduciéndolo a `/cmd_vel`; y el control por gestos nativo de ROS, que usa la webcam y MediaPipe para producir *gesture_to_cmd_vel*. Siempre bajo la regla de un único dueño de `/cmd_vel`.

5.3 Fase 3 — Percepción simulada

La cámara simulada del robot publica en `/camera/image_raw`. Sobre ella se portó la detección de balón por color (*ball_detector* en *footbot_perception*), determinista y ajustada al balón naranja simulado, y se incorporó la visión de fútbol con YOLO en *footbot_soccer_vision*.

5.4 Fase 4 — Comportamientos autónomos deterministas

Es el corazón del trabajo: dos comportamientos de fútbol contruidos sobre el patrón *percepción* → *estimador de estado* → *FSM* → un único dueño de `/cmd_vel`. Se detallan en la Sección 6.

5.5 Fase 5 — Datasets y entrenamiento de visión

Se construyó la tubería completa de datos para YOLO: captura de imágenes desde la escena (*image_capture*), etiquetado en Label Studio, aumento conservador del *dataset*, preparación

y validación de las particiones, y entrenamiento/evaluación del detector de `ball + goal`. Se detalla en la Sección 7.

5.6 Mundos y escenarios

Se crearon mundos para cada necesidad: pruebas de cámara, control de balón de un escenario, prueba multi-carril (tres escenarios aislados en un mismo mundo), escena de *Llegar a la portería* (robot + balón dinámico + portería) y el campo de fútbol completo con paredes, porterías, balón central y equipos en espejo (Fig. 9).

6. Comportamientos autónomos en detalle

Ambos comportamientos comparten una filosofía **determinista**: la percepción alimenta a un estimador de estado, el estimador alimenta a una FSM, y exactamente un controlador es dueño de `/cmd_vel`. Las *skills* sólo producen comandos `Twist` acotados para el estado actual; la FSM es la única dueña de las transiciones.

6.1 Control de balón (Ball Control)

Es el primer comportamiento autónomo: encontrar, aproximarse, hacer contacto y mantener el balón naranja en una zona de control frontal (Fig. 3). El detector HSV produce `/ball_detection`; el `ball_state_estimator` convierte la detección `Detection2D` en un `BallState`; y el `ball_control_fsm` recorre estados como `SEARCH_BALL`, `ALIGN_TO_BALL`, `APPROACH_BALL`, `CONTACT_BALL`, `CONTROL_BALL`, `ROTATE_WITH_BALL`, `RECOVER_BALL` y `STOP_SAFE`. El estimador usa el radio aparente del balón y el campo de visión de la cámara para estimar un rango aproximado: apropiado para simulación, sin pretender ser un estimador de pose 3D calibrado.

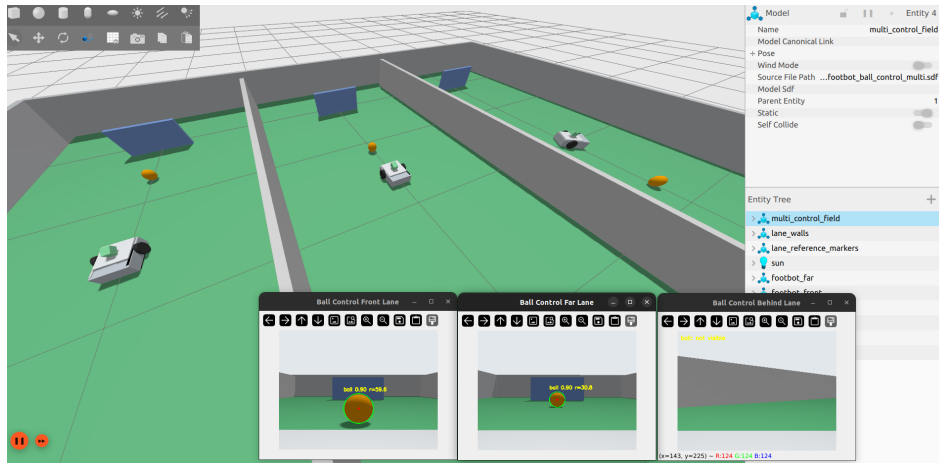


Figura 3: Comportamiento de Control de balón ejecutándose en Gazebo, con el visor de depuración que muestra el balón naranja detectado y la realimentación de estado.

¿Cómo mantiene el balón centrado? (paso a paso). La idea es sencilla: el robot **solo ve el balón en la imagen** de la cámara, no conoce su posición real. Que el balón esté *centrado* significa que aparece en el **centro horizontal** de la imagen. El robot mide cuánto se desvía de ese centro y gira para corregirlo, igual que cuando caminamos hacia algo que estamos mirando.

Paso 1 — Medir qué tan descentrado está. Si el balón aparece en la columna u (en píxeles) de una imagen de ancho W , el centro está en $W/2$. Esa desviación se traduce a un

ángulo de rumbo θ (cuánto tendría que girar el robot para mirarlo de frente):

$$\theta = \arctan\left(\frac{u - W/2}{f}\right). \quad (1)$$

Si el balón está justo en el centro, $\theta = 0$; si está a la derecha, $\theta > 0$; si está a la izquierda, $\theta < 0$. (El factor f solo traduce píxeles a ángulos según el lente de la cámara.)

Paso 2 — Estimar a qué distancia está. Cuanto más grande se ve el balón, más cerca está. A partir de su radio aparente r (px) y su radio real conocido R :

$$d = \frac{f R}{r}. \quad (2)$$

Paso 3 — Girar para centrarlo. El robot gira con una velocidad angular ω **proporcional al error**: si el balón está muy descentrado, corrige rápido; cuando ya está casi centrado, gira suave:

$$\omega = -K_p \theta. \quad (3)$$

Aquí K_p es solo “cuánto” corrige, y el signo negativo significa “gira hacia el lado que reduce el error”. Además, el comando se **suaviza** (se mezcla con el anterior) para que el giro no sea brusco y se **limita** a una velocidad máxima de seguridad.

Paso 4 — Decidir que “ya lo controla”. No basta con un instante: el robot da el balón por *controlado* solo si lleva un tiempo T_{hold} con el balón **centrado y cerca** a la vez:

$$|\theta| \leq \theta_{\text{tol}} \quad \text{y} \quad d \leq d_{\text{ctrl}} \quad \text{durante} \geq T_{\text{hold}}. \quad (4)$$

En la simulación se usan $K_p = 0,9$, $\theta_{\text{tol}} = 0,18$ rad ($\approx 10^\circ$), $d_{\text{ctrl}} = 0,20$ m y $T_{\text{hold}} = 0,35$ s.

6.2 Llegar a la portería (Reach Goal)

Es el comportamiento de producción: empujar el balón hacia una portería visible usando la cámara del robot. Un detector YOLO de `ball + goal` publica `/soccer/detections`; el `ball_goal_state_estimator` produce un `BallGoalState` derivado únicamente de la imagen; y el `reach_goal_fsm` es el único dueño de `/cmd_vel`. Cada decisión de control está guiada por la percepción: el comportamiento **nunca** lee las poses reales (*ground truth*) de Gazebo para navegar. La Tabla 3 resume sus estados.

Tabla 3: Estados de la FSM de *Llegar a la portería*.

Estado	Descripción
SEARCH_BALL	Sin balón a la vista; rota para escanear.
APPROACH_BALL	Balón visible; conduce hacia él (frena al crecer).
CONTROL_BALL	Balón en la zona de control frontal; lo mantiene centrado.
SEARCH_GOAL	Balón controlado, portería no vista; rota sosteniendo el balón.
ALIGN_BALL_TO_GOAL	Balón y portería visibles pero desalineados; gira suave con el balón.
DRIBBLE_TO_GOAL	Balón alineado con la portería; avanza.
COMMIT_TO_GOAL	Respaldo a corta distancia; empuja despacio sólo centrando el balón.
RECOVER_BALL	Control perdido; retrocede y readquiere el balón.
STOP_SAFE	Percepción obsoleta o parada de emergencia; velocidad cero.
GOAL_SCORED	El árbitro de simulación detectó gol; velocidad cero permanente.

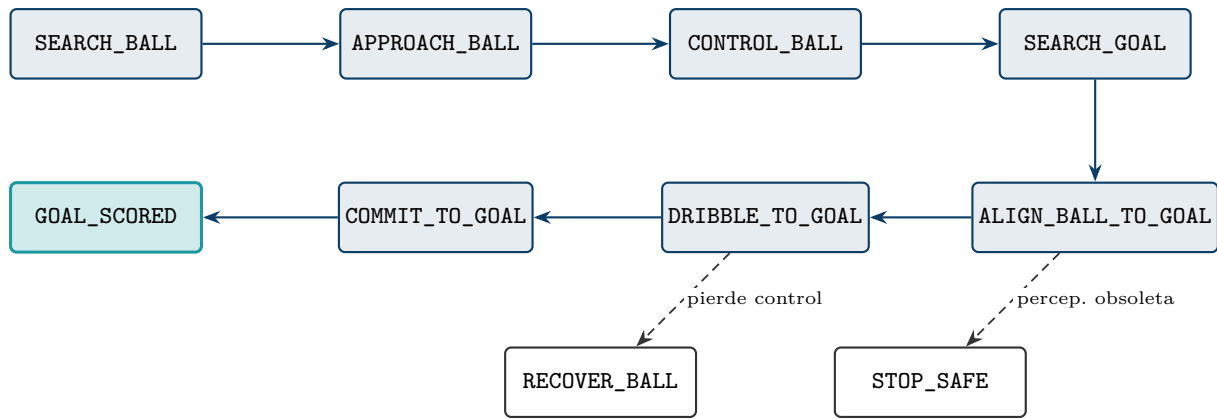


Figura 4: Esquema de la FSM de *Llegar a la portería*. En azul, el camino principal (de buscar el balón a marcar, con **GOAL_SCORED** resaltado como estado terminal); en líneas discontinuas, las salidas de recuperación y de seguridad. Cualquier estado de posesión puede caer a **RECOVER_BALL** al perder el control, y la percepción obsoleta fuerza **STOP_SAFE**; ambos retoman la búsqueda al recuperarse.

La Figura 4 resume gráficamente estas transiciones. Dos mecanismos hacen robusto el comportamiento. Primero, una **memoria temporal de la portería**: cuando el robot se acerca a la boca y la portería desaparece del cuadro, el estimador reutiliza brevemente la última orientación válida, evitando recaer en **SEARCH_GOAL** por una pérdida puntual de YOLO. Segundo, el estado **COMMIT_TO_GOAL**, un respaldo *sin reentrenamiento* que, tras **DRIBBLE_TO_GOAL**, mantiene un empuje lento centrado en el balón hasta que el árbitro reporta el gol.

Detección de gol = lógica de árbitro, no percepción del robot

La puntuación usa un monitor de simulación (`reach_goal_score_monitor`) que observa el *stream* de poses del mundo de Gazebo y publica `/soccer/goal_scored` cuando el balón permanece dentro de la zona de gol el tiempo suficiente. Esto permite **detener** el episodio limpiamente tras un gol **sin** dar al robot información privilegiada para navegar.

7. Percepción y *datasets*

7.1 Cámaras y detección

La cámara simulada del robot publica en `/camera/image_raw` (y `/camera/camera_info`); el campo de fútbol añade además una cámara cenital en `/soccer/camera/image_raw`. Sobre estas imágenes operan dos familias de detectores: el detector HSV determinista del balón (`ball_detector`) y los detectores YOLO de `footbot_soccer_vision`, que emiten `vision_msgs/Detection2D`.

El control por gestos, heredado del modo manual del robot real, también se reprodujo de forma nativa en ROS con MediaPipe (Fig. 5).

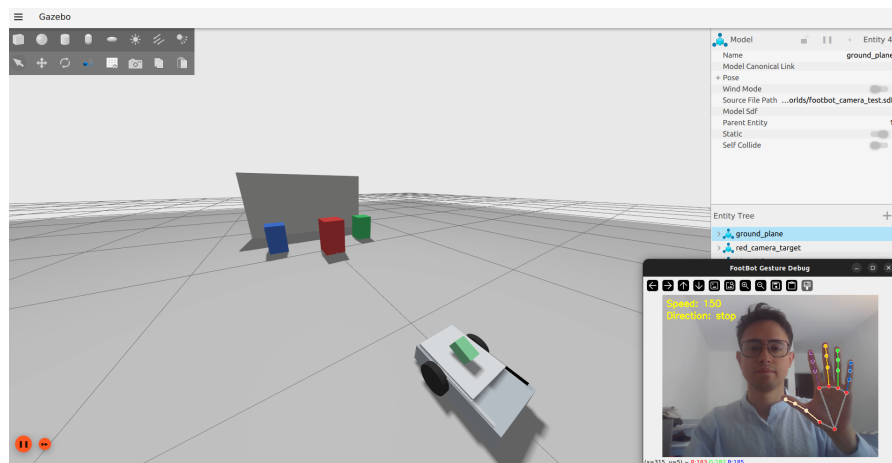


Figura 5: Control por gestos nativo de ROS: vista de depuración con los *landmarks* de la mano de MediaPipe, la dirección detectada y la realimentación de velocidad.

7.2 Tubería de *dataset* y entrenamiento

Para *Llegar a la portería* se construyó una tubería de datos completa y reproducible:

1. **Captura:** `image_capture` guarda imágenes de la escena desde `/camera/image_raw`.
2. **Etiquetado:** las clases de fútbol (`ball`, `goal`, `opponent`) se etiquetan en Label Studio (Fig. 6) y se exportan en formato YOLO.
3. **Aumento:** un script genera copias rotadas/oscurcidas de forma conservadora.
4. **Preparación y validación:** se preparan y validan las particiones `train/val` del *dataset* de `ball` + `goal`.
5. **Entrenamiento y evaluación:** herramientas de entrenamiento, evaluación y predicción de YOLO cierran el ciclo.



Figura 6: Etiquetado de objetos de fútbol en Label Studio para el entrenamiento de YOLO, con cajas para `ball`, `goal` y `opponent`.

Desempeño del detector. Sobre el conjunto de validación, el detector YOLO de `ball` y `goal` **funciona muy bien**: la matriz de confusión del entrenamiento (Figs. 7 y 8) muestra el balón clasificado correctamente en el 100 % de los casos y la portería en el 90 % (con un 10 %

de porterías no detectadas, confundidas con el fondo). Es decir, **en las imágenes del *dataset* la percepción es fiable**. La limitación real no está en el *dataset*, sino **en despliegue, cuando el robot se acerca demasiado a la portería**: a corta distancia la portería deja de caber en el cuadro de la cámara y el detector ya no la reconoce de forma continua, lo que degrada el remate (Sección 8.1).

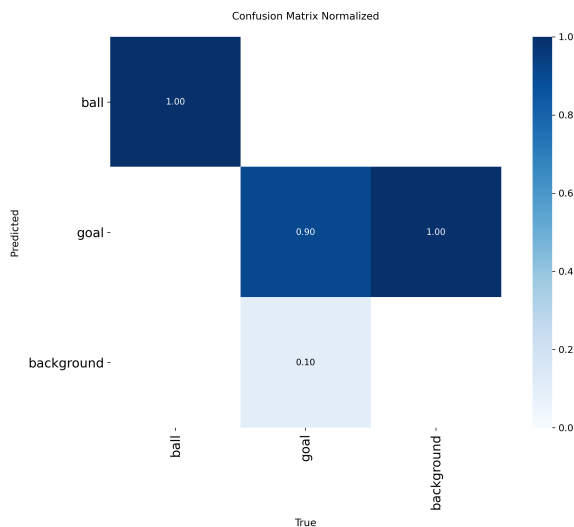


Figura 7: Matriz de confusión **normalizada** del entrenamiento de YOLO (**ball + goal**) sobre el conjunto de validación: el balón se clasifica correctamente el 100 % de las veces y la portería el 90 %. En el *dataset* la percepción es fiable.

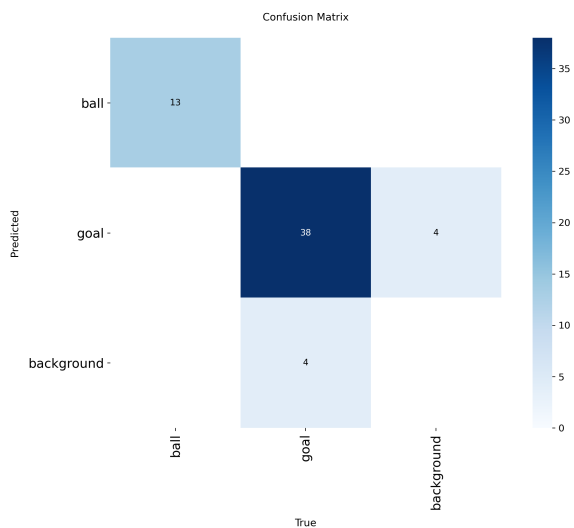


Figura 8: Matriz de confusión en **conteos absolutos** (mismos datos de la Fig. 7). El problema de detección no aparece en el *dataset*, sino sólo a muy corta distancia durante la simulación (Sección 8.1).

8. Resultados y estado actual

A la fecha de este reporte, la capa de simulación tiene implementado y funcionando:

- Modelo del robot FootBot y generación en Gazebo.

- Movimiento por tracción diferencial vía `/cmd_vel`.
- Puente de compatibilidad HTTP `/move` (ESP32) hacia `/cmd_vel`.
- Cámara simulada del robot y control por gestos nativo de ROS.
- Detección HSV del balón y seguidor de balón simple.
- **Control de balón** determinista con orquestación por FSM.
- **Llegar a la portería** determinista, con percepción YOLO de balón + portería, memoria temporal de la portería y un monitor de puntuación de simulación.
- Herramientas de *dataset* (validación, preparación, aumento) y de entrenamiento/evaluación de YOLO.
- Mundos de campo de fútbol y multi-escenario.

Verificación

El *workspace* compila con *colcon* y la suite de pruebas del paquete *footbot_soccer_behavior* pasa correctamente (**41 pruebas, 0 fallos, 3 omitidas**), cubriendo la lógica de los estimadores de estado y de las FSM.

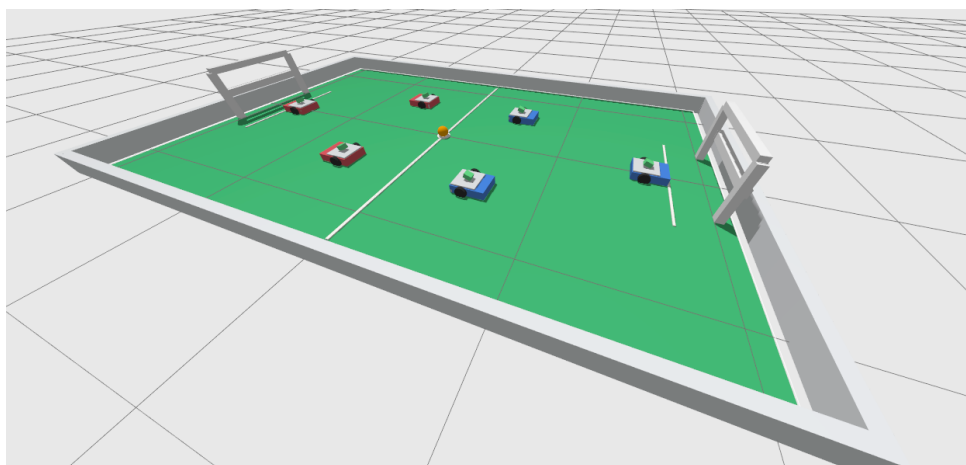


Figura 9: Mundo de campo de fútbol completo: paredes de límite, dos porterías, un balón central y equipos de robots en espejo.

8.1 Análisis experimental: Llegar a la portería (5 episodios)

Se ejecutó una tanda de **cinco episodios** del comportamiento *Llegar a la portería*, registrando los *topics* con *rosvbag2* y procesándolos de forma automática (los *scripts* de medición y graficado viven en `reports/footbot-simulation-es/metrics/`). La Figura 10 muestra una vista cenital del campo —la portería arriba y el robot partiendo desde abajo— con la trayectoria combinada de los cinco episodios y los mapas de densidad de permanencia del robot y del balón.

El robot **controla el balón de forma rápida y consistente** (primer control en ≈ 9 s en todos los episodios), pero **rematar resulta costoso**: sólo **1 de 5** episodios terminó en gol (20 % de éxito) y, en ese caso, tardó cerca de **269 s**. Tanto las trayectorias como los mapas de calor muestran que el robot y el balón **acumulan la mayor parte del tiempo justo frente**

a la boca de la portería (zona superior, en torno a $x \approx 1,4$ m), sin completar el remate.

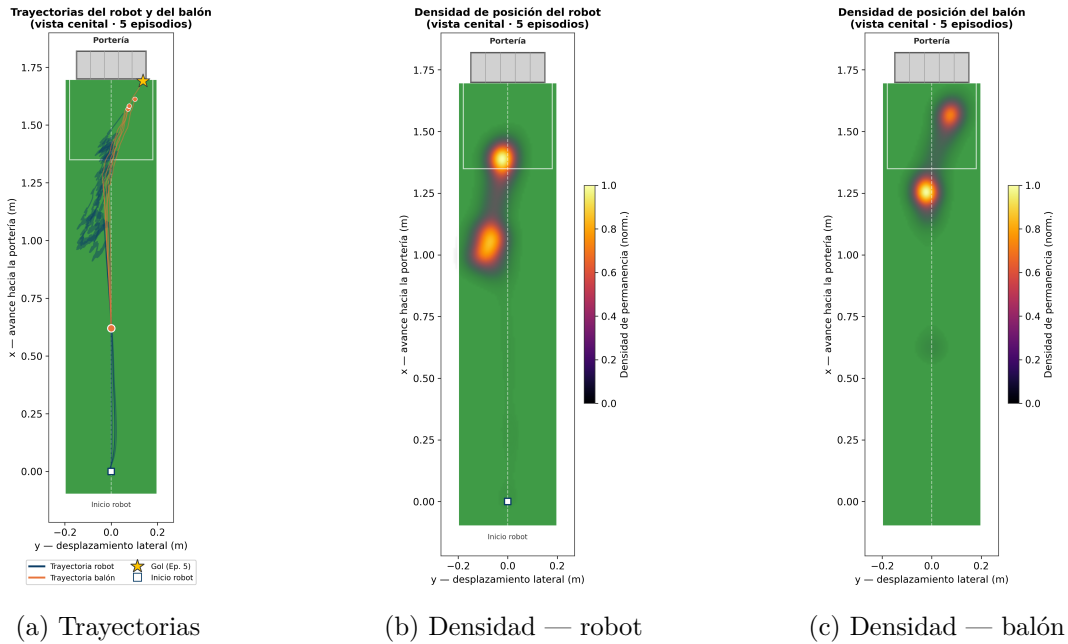


Figura 10: Análisis cenital de cinco episodios de *Llegar a la portería* (la portería está arriba; el robot parte desde abajo y avanza hacia ella). (a) Trayectorias combinadas del robot (azul) y del balón (naranja); la estrella marca el gol del episodio 5. (b) Densidad de permanencia del robot y (c) del balón: ambos concentran el tiempo *frente a la boca de la portería*, donde la detección visual de la portería se vuelve intermitente a corta distancia.

Causa principal: detección de la portería a corta distancia. El comportamiento se guía por la detección visual de la portería con YOLO. Cuando el robot se **acerca mucho**, la portería **deja de caber por completo en el cuadro de la cámara** y el detector ya no la reconoce con precisión ni de forma continua: las detecciones se vuelven intermitentes y la FSM **pierde la referencia de la portería** repetidamente. En consecuencia, el robot alterna entre aproximarse, perder el objetivo y **recuperar/realinear**, lo que dispara el tiempo de remate. La *memoria temporal de la portería* (Sección 6) atenúa las desapariciones breves, pero a muy corta distancia no es suficiente.

El **reparto de tiempo de la FSM** (medido sobre `/soccer/reach_goal_fsm_state`) confirma el patrón: **RECOVER BALL** concentra alrededor del **33 %** del tiempo y **APPROACH BALL** otro **26 %**, mientras que **DRIBBLE TO GOAL** —el avance de remate— apenas acumula tiempo. Es decir, el robot pasa la mayor parte del episodio *recuperando y reaproximándose* en vez de empujar el balón al fondo. Más revelador aún: el estado de **respaldo a corta distancia COMMIT TO GOAL** (Tabla 3) —diseñado precisamente para empujar *sin* depender de ver la portería— **no se activó en ninguno de los cinco episodios**; el robot siguió dependiendo de la detección continua de la portería, que es justo lo que falla de cerca.

9. Trabajo futuro

La hoja de ruta mantiene el principio determinista como base estable y añade tácticas sólo cuando son medibles:

- **Detección de la portería a corta distancia (prioritario):** resolver la pérdida de detección cuando el robot está muy cerca y la portería no cabe en el cuadro (Sección 8.1).

Posibles soluciones: (i) ampliar el *dataset* con vistas de portería a muy corta distancia y parciales, y reentrenar YOLO; (ii) detectar *partes* de la portería (postes, línea de meta) en lugar de la caja completa; (iii) usar una cámara de mayor campo de visión (FOV); (iv) activar de forma fiable el estado de respaldo COMMIT_TO_GOAL —un *modo de remate por proximidad* que empuja el balón con la última pose conocida de la portería y odometría/*dead reckoning*, sin exigir detección continua—; y (v) reforzar la memoria temporal de la portería con anclaje geométrico.

- **Interacción con oponentes:** detectar a un oponente con el balón y disputar la posesión sin desestabilizar los comportamientos base; estados de FSM para observar, aproximarse, disputar y recuperar.
- **Juego en equipo:** soportar tres robots por bando con roles (portero, defensor, atacante), *namespaces* por robot para evitar colisiones de topics, y comportamiento de pase.
- **Líneas de investigación:** FSM heurísticas y árboles de comportamiento; planificación de horizonte corto (MCTS/SPO) y, sólo cuando la simulación tenga *reset*, puntuación y métricas estables, aprendizaje por refuerzo (RL/MARL).

10. Conclusiones

La migración a ROS 2 + Gazebo convirtió al FootBot de un banco de pruebas físico, frágil y de iteración lenta, en un entorno de desarrollo **reproducible, seguro y escalable**, sin desechar el trabajo previo: el robot ESP32 y sus aplicaciones siguen como referencia y su protocolo HTTP /move se conserva mediante un puente. Sobre esa base se construyeron dos comportamientos de fútbol deterministas —Control de balón y Llegar a la portería— bajo un patrón claro de *percepción* → *estado* → *FSM* → *un único dueño de /cmd_vel*, junto con una tubería completa de *datasets* y entrenamiento de visión. El resultado es una plataforma sólida para las siguientes etapas: oponentes, juego en equipo y experimentos tácticos.

Referencias

- Documentación técnica del proyecto: `simulation/docs/` (índice, arquitectura, modos, control de balón, llegar a la portería, percepción y *datasets*, mundos y escenarios).
- ROS 2 Humble — Instalación en Ubuntu: <https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html>
- Gazebo Fortress — Documentación: <https://gazebo-sim.org/docs/fortress/install>
- Ultralytics YOLO: <https://github.com/ultralytics/ultralytics>
- MediaPipe (Google): <https://github.com/google/mediapipe>